

Django ORM, Query Optimization, Database Indexing & DBMS

Complete Notes --

Part 1 -- select_related()

1. select_related() ?

select_related() Django ORM- **query optimization technique**

ForeignKey OneToOneField relationship- related object- **SQL JOIN- database query-**

Example

```
javascript.snippet JAVASCRIPT

class Author(models.Model):    name = models.CharField(max_length=100)
class Book(models.Model):
    title = models.CharField(max_length=100)    author = models.ForeignKey(
        Author, on_delete=models.CASCADE    )
```

Book Author ForeignKey relationship

Without select_related():

```
javascript.snippet JAVASCRIPT

books = Book.objects.all()
for book in books:    print(book.title, book.author.name)
```

book.author access query

With select_related()

```
javascript.snippet JAVASCRIPT

books = Book.objects.select_related("author")
for book in books:    print(book.title,
    book.author.name)
```

Django SQL JOIN Book Author- related data

2. select_related() ?

relationship:

```
javascript.snippet
```

JAVASCRIPT

```
ForeignKeyOneToOneField
```

related object- data

Example

```
javascript.snippet
```

JAVASCRIPT

```
Book.objects.select_related("author")
```

Part 2 -- prefetch_related()

3. prefetch_related() ?

prefetch_related() related objects efficiently load

:

```
javascript.snippet
```

JAVASCRIPT

```
ManyToManyFieldReverse ForeignKey
```

useful

select_related() SQL JOIN , prefetch_related() **query related objects fetch** ,
Django Python side- objects- map

4. Example -- Reverse ForeignKey

```
javascript.snippet JAVASCRIPT

class Author(models.Model):    name = models.CharField(max_length=100)
class Book(models
Model):    title = models.CharField(max_length=100)    author = models.ForeignKey(
Author,        on_delete=models.CASCADE,        related_name="books"    )
```

```
javascript.snippet JAVASCRIPT

authors = Author.objects.prefetch_related("books")
```

```
javascript.snippet JAVASCRIPT

for author in authors:    print(author.name)    for book in author.books.all():
print
(book.title)
```

Django Author query related Book objects- query

5. select_related() prefetch_related()

	select_related()	prefetch_related()
Main use	FK / OneToOne	M2M / Reverse FK
Technique	SQL JOIN	Separate query + Python mapping
Query	1	2
Best for	Single-valued relationship	Multi-valued relationship
Example	Book Author	Author Books

```
javascript.snippet JAVASCRIPT

ForeignKey / OneToOne    select_related()
prefetch_related        ManyToMany / Reverse ForeignKey
()
```

Part 3 -- Other Query Optimization Methods

6. only()

only() QuerySet- field initially load

```
javascript.snippet
```

JAVASCRIPT

```
books = Book.objects.only("title")
```

title field initially load

only() " field- database "--

Deferred fields access Django query

Example:

```
javascript.snippet
```

JAVASCRIPT

```
books = Book.objects.only("title")for book in books:    print(book.title)
```

efficient title

:

```
javascript.snippet
```

JAVASCRIPT

```
print(book.description)
```

deferred field load query

7. defer()

defer() field- initial loading

```
javascript.snippet
```

JAVASCRIPT

```
books = Book.objects.defer("description")
```

description initially load

field, :

```
javascript.snippet JAVASCRIPT

TextFieldFileField

, field immediately
```

8. only() defer()

```
javascript.snippet JAVASCRIPT

only() specify defer() specify
```

Part 4 -- annotate()

9. annotate() ?

annotate() QuerySet- object- **calculated value**

Example:

```
javascript.snippet JAVASCRIPT

from django.db.models import Count
authors = Author.objects.annotate(
    book_count=Count("books"))

:
```

```
javascript.snippet JAVASCRIPT

for author in authors:
    print(author.name, author.book_count)

:
```

```
javascript.snippet JAVASCRIPT

book_count

calculated value
```

10. annotate() Example

:

```
javascript.snippet JAVASCRIPT
Author A  5 booksAuthor B  3 booksAuthor C  8 books
```

:

```
javascript.snippet JAVASCRIPT
authors = Author.objects.annotate(    book_count=Count("books"))
```

:

```
javascript.snippet JAVASCRIPT
Author A  book_count =  B  book_count =  C  book_count = 8
```

Part 5 -- aggregate()

11. aggregate() ?

aggregate() QuerySet- **summary/calculation**

Example:

```
javascript.snippet JAVASCRIPT
from django.db.models import Avgresult = Book.objects.aggregate(    Avg("price"))
```

Result:

```
javascript.snippet JAVASCRIPT
{    "price__avg": 245.66}
```

Common aggregate functions:

```
javascript.snippet JAVASCRIPT
CountSumAvgMinMax
```

12. annotate() aggregate()

annotate()

object- calculated value:

```
javascript.snippet JAVASCRIPT
Author.objects.annotate(    book_count=Count("books"))
```

aggregate()

QuerySet- summary:

```
javascript.snippet JAVASCRIPT
Book.objects.aggregate(    Avg("price"))
```

:

```
javascript.snippet JAVASCRIPT
annotate    object- calculationaggregate    QuerySet- summary
```

Part 6 -- values() values_list()

13. values()

values() QuerySet- result dictionary

```
javascript.snippet JAVASCRIPT
Book.objects.values(    "title",    "author__name")
```

Example result:

```
javascript.snippet JAVASCRIPT
[ { "title": "Django Basics", "author__name": "Faruk" } ]
```

14. values_list()

values_list() tuple result

```
javascript.snippet JAVASCRIPT
Book.objects.values_list("title", "price")
```

Result:

```
javascript.snippet JAVASCRIPT
[ ("Django Basics", 500), ("Python", 400)]
```

field :

```
javascript.snippet JAVASCRIPT
Book.objects.values_list("title", flat=True)
```

Result:

```
javascript.snippet JAVASCRIPT
[ "Django Basics", "Python"]
```

Part 7 -- Basic QuerySet Methods

15. all()

object- QuerySet:

```
javascript.snippet JAVASCRIPT
Book.objects.all()
```

16. filter()

condition object:

 javascript.snippet

JAVASCRIPT

```
Book.objects.filter(    price__gt=500)
```

17. exclude()

condition :

 javascript.snippet

JAVASCRIPT

```
Book.objects.exclude(    price=0)
```

18. get()

object retrieve

 javascript.snippet

JAVASCRIPT

```
book = Book.objects.get(id=1)
```

object :

 javascript.snippet

JAVASCRIPT

```
DoesNotExist
```

object :

 javascript.snippet

JAVASCRIPT

```
MultipleObjectsReturned
```

get() condition- ideally object return

19. order_by()

Ascending:

```
javascript.snippet JAVASCRIPT
Book.objects.order_by("price")
```

Descending:

```
javascript.snippet JAVASCRIPT
Book.objects.order_by("-price")
```

Multiple fields:

```
javascript.snippet JAVASCRIPT
Book.objects.order_by("author", "-price")
```

20. reverse()

ordering- ordering

```
javascript.snippet JAVASCRIPT
books = Book.objects.order_by("title").reverse()
```

21. distinct()

Duplicate result unique result

```
javascript.snippet JAVASCRIPT
Book.objects.values("author").distinct()
```

Part 8 -- Result Checking Methods

22. exists()

QuerySet- object check

```
javascript.snippet JAVASCRIPT
if Book.objects.filter(    title="Django").exists():    print("Book found")
```

Result:

```
javascript.snippet JAVASCRIPT
True / False
```

existence check , count() appropriate

23. count()

QuerySet- object :

```
javascript.snippet JAVASCRIPT
count = Book.objects.count()
```

24. first()

object return

```
javascript.snippet JAVASCRIPT
book = Book.objects.first()
```

object :

```
javascript.snippet JAVASCRIPT
None
```

return

25. last()

object return

```
javascript.snippet
```

JAVASCRIPT

```
book = Book.objects.last()
```

```
:
```

```
javascript.snippet
```

JAVASCRIPT

```
None
```

26. earliest()

field object:

```
javascript.snippet
```

JAVASCRIPT

```
book = Book.objects.earliest("created_at")
```

27. latest()

field object:

```
javascript.snippet
```

JAVASCRIPT

```
book = Book.objects.latest("created_at")
```

28. none()

Empty QuerySet return

```
javascript.snippet
```

JAVASCRIPT

```
books = Book.objects.none()
```

database- actual data fetch empty QuerySet

29. in_bulk()

object dictionary return

```
javascript.snippet
```

JAVASCRIPT

```
books = Book.objects.in_bulk([1, 2, 3])
```

Result conceptually:

```
javascript.snippet
```

JAVASCRIPT

```
{ 1: <Book object>, 2: <Book object>, 3: <Book object>}
```

Part 9 -- Create / Update / Delete

30. create()

object create object return

```
javascript.snippet
```

JAVASCRIPT

```
book = Book.objects.create(title="Django", price=500)
```

31. update()

matching row update

```
javascript.snippet
```

JAVASCRIPT

```
Book.objects.filter(price=500).update(price=600)
```

update() updated row- return

32. delete()

QuerySet- matching objects delete

```
Book.objects.filter(price=0).delete()
```

33. bulk_create()

object create

```
Book.objects.bulk_create([
    Book(title="Book 1"),
    Book(title="Book 2"),
    Book(title="Book 3"),])
```

object insert useful

34. bulk_update()

existing object- field update

```
books = Book.objects.filter(category="django")
for book in books:
    book.price += 100
Book.objects.bulk_update(books, ["price"])
```

35. get_or_create()

Object return

create

```
book, created = Book.objects.get_or_create(title="Django")
```

created:

```
javascript.snippet JAVASCRIPT

True    object    False    object
```

36. update_or_create()

Object update

create

```
javascript.snippet JAVASCRIPT

book, created = Book.objects.update_or_create(    title="Django
",    defaults={    "
price": 500    })
```

Part 10 -- Date/Time Query Methods

37. dates()

DateField/DateTimeField unique dates

```
javascript.snippet JAVASCRIPT

Book.objects.dates(    "created_at",    "day")
```

38. datetimes()

DateTimeField unique datetime values

```
javascript.snippet JAVASCRIPT

Book.objects.datetimes(    "created_at",    "day")
```

Part 11 -- Database Locking

39. select_for_update()

select_for_update() database transaction- selected rows- **row-level lock**

:

```
javascript.snippet JAVASCRIPT
transaction.atomic()
```

Example:

```
javascript.snippet JAVASCRIPT
from django.db import transaction
with transaction.atomic():
    book = Book.objects.select_for_update().get(id=1)
    book.stock -= 1
    book.save()
```

concurrent transaction- race condition

select_for_update() database backend- database- locking support-

Part 12 -- Raw SQL

40. raw()

Raw SQL model objects retrieve

```
javascript.snippet JAVASCRIPT
books = Book.objects.raw("SELECT * FROM app_book")
```

Django ORM query- raw()

ORM

41. extra()

extra() /deprecated API code-

Alternative Django ORM expressions

:

```
annotate()F()Q()Subquery()Exists()Case()When()
```

Part 13 -- QuerySet Lazy Evaluation

42. Django QuerySet Lazy ?

Django QuerySet lazy

:

```
books = Book.objects.filter( price__gt=500)
```

statement database query execute

QuerySet evaluate database query execute

Example:

```
for book in books:      print(book.title)
```

QuerySet evaluate

43. Common Evaluation Triggers

QuerySet evaluate :

```
for looplist(queryset)len(queryset)bool(queryset)iteration
```

Important correction

values() values_list() "database hit trigger"

QuerySet ; QuerySet- evaluate query execute

aggregate() query execute result

Part 14 -- Safe First Object

44. List object

:

```
javascript.snippet JAVASCRIPT
books = list(Book.objects.filter(is_published=True))
```

:

```
javascript.snippet JAVASCRIPT
books[0]
```

list empty :

```
javascript.snippet JAVASCRIPT
IndexError
```

45. Safe Way

```
javascript.snippet JAVASCRIPT
first_book = books[0] if books else None
```

:

```
javascript.snippet JAVASCRIPT
first_book = next(iter(books), None)
```

46. QuerySet- .first()

:

```
javascript.snippet JAVASCRIPT
first_book = ( Book.objects .filter(is_published=True) .first())
```

object :

```
javascript.snippet JAVASCRIPT
None
```

return

Part 15 -- Database Indexing

47. Database Indexing ?

Database- data index

:

index topic

Database index-

48. Index ?

Index query

:

```
javascript.snippet JAVASCRIPT
WHEREORDER BYJOINUNIQUE lookup
```

query pattern- index useful

index useful database query pattern-

49. Index

database- row

:

```
javascript.snippet JAVASCRIPT

Table Scan
```

table- expensive

50. Index

Database index matching rows locate

relational database- index- B-tree/B+tree- data structure

Important correction

:

"Index direct memory location data "

:

Index database- matching row locate

Part 16 -- Django db_index

51. db_index=True

field- index :

```
javascript.snippet JAVASCRIPT

email = models.EmailField( db_index=True)
```

Migration apply database- index

52. Meta.indexes

Multiple/composite index :

```
javascript.snippet JAVASCRIPT

class Student(models.Model):
    city = models.CharField(max_length=100)
    age = models.IntegerField()
    class Meta:
        indexes = [
            models.Index(
                fields=["city", "age"]
            ),
        ]
```

composite index

53. db_index Meta.indexes

	db_index=True	Meta.indexes
Scope	field	field
Composite index		
Location	Field definition	Model Meta
Flexibility		

Part 17 -- Composite Index

54. Composite Index ?

field- index:

```
javascript.snippet JAVASCRIPT

models.Index(fields=["city", "age"])
```

composite/multi-column index

application- query pattern :

```
javascript.snippet JAVASCRIPT

Student.objects.filter(city="Dhaka", age=20)
```

index useful

index design actual query patterns database query plan

55. Index ?

field- frequently query :

```
javascript.snippet
```

JAVASCRIPT

```
filter()order_by()joinunique lookup
```

field- index useful

ForeignKey field- Django database-level indexing- , exact behavior custom indexes-

Django/database version schema

56. Index Avoid ?

Index- cost

index:

```
javascript.snippet
```

JAVASCRIPT

```
Storage INSERT cost UPDATE cost DELETE cost
```

field- index

57. Index-

```
javascript.snippet
```

JAVASCRIPT

```
v Query performance improve v Filtering v Sorting- query optimize v Large table-  
lookup  
efficient
```

58. Index-

storage Write operation- overhead index maintenance

Part 18 -- OTP Query Optimization

59. Multiple Active OTP Check

user- unused OTP

latest OTP active OTP check :

```
active_otps = (      cls.objects      .select_for_update()      .filter(      user=user,
is_used=False      )      .order_by("-created_at"))
```

:

```
for otp_obj in active_otps:      if hmac.compare_digest(      cls.hash_otp(
otp_code
,      otp_obj.otp_salt      ),      otp_obj.otp_hash      ):      # OTP
matched
...      break
```

60. OTP Code- Important Points

select_for_update() :

```
with transaction.atomic():
```

query

Example:

```
javascript.snippet JAVASCRIPT

from django.db import transaction
with transaction.atomic():
    active_otps = (
        cls
        .objects
        .select_for_update()
        .filter(
            user=user,
            is_used
            =False
        )
        .order_by("-created_at")
    )
    for otp_obj in active_otps:

        if hmac.compare_digest(
            cls.hash_otp(
                otp_code,
                otp_obj.otp_salt
            ),
            otp_obj.otp_hash
        ):
            ...

        break
```

?

Concurrent request OTP process race condition , row locking useful

exact transaction/locking strategy application- OTP lifecycle-

Part 19 -- DBMS: Integrity

61. Integrity ?

Database- data :

```
javascript.snippet JAVASCRIPT

CorrectValidConsistent with defined rules
```

, **Data Integrity**

Example:

```
javascript.snippet JAVASCRIPT

Age = -10
```

application- rule age negative invalid , integrity violation

62. Types of Integrity

:

```
javascript.snippet JAVASCRIPT
Entity IntegrityReferential IntegrityDomain Integrity
```

63. Entity Integrity

Entity integrity primary key:

```
javascript.snippet JAVASCRIPT
NULL Unique
```

row- uniquely identify primary key

64. Referential Integrity

Foreign key valid parent record- reference ,

Example:

```
javascript.snippet JAVASCRIPT
Department Student
```

Student- department_id ideally existing Department- valid key reference

65. Domain Integrity

Field- value :

```
javascript.snippet JAVASCRIPT
Data typeRangeFormatAllowed values
```

Example:

```
javascript.snippet JAVASCRIPT
Age = -10
```

age- valid range:

```
javascript.snippet JAVASCRIPT
0-150
```

, -10 invalid

Part 20 -- Consistency

66. Consistency ?

ACID- context- consistency transaction- database defined rules/constraints **valid state-**

Example:

Bank transfer:

```
javascript.snippet JAVASCRIPT
A = 200 transferA = 700
```

Transaction- database- rules valid state

Part 21 -- Constraint

67. Constraint ?

Database- data rules , constraints

Common constraints:

```
javascript.snippet JAVASCRIPT
PRIMARY KEY FOREIGN KEY UNIQUE NOT NULL CHECK
```

68. Primary Key

Primary key:

```
javascript.snippet JAVASCRIPT
UniqueNot NULL
```

Example:

```
javascript.snippet JAVASCRIPT
id INTEGER PRIMARY KEY
```

69. Foreign Key

table- record table- record- relationship

Example:

```
javascript.snippet JAVASCRIPT
author = models.ForeignKey( Author, on_delete=models.CASCADE)
```

70. Unique Constraint

value duplicate prevent

Django:

```
javascript.snippet JAVASCRIPT
email = models.EmailField( unique=True)
```

71. NOT NULL

Field- NULL value prevent

Database-level NOT NULL Django- blank=False

:

```
null=False database NULL behaviorblank=False validation/form behavior
```

72. CHECK Constraint

condition enforce

Django example:

```
from django.db import models
class Student(models.Model):
    age = models.IntegerField()

    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=
                =models.Q(age__gte=0),
                name="age_gte_0"
            )
        ]
```

age negative rule

Part 22 -- Redundancy

73. Redundancy ?

information database- store data redundancy

Example:

```
Student tableName = FarukPhone = tableName = FarukPhone =
```

74. Redundancy-

```
Storage wasteUpdate anomalyInsert anomalyDelete anomalyData inconsistency
```

Normalization redundancy

Part 23 -- Inconsistency

75. Inconsistency ?

data value data inconsistency

Example:

```
javascript.snippet JAVASCRIPT
Table A:Phone = B:Phone =
```

phone number represent , database inconsistent

76. Redundancy Inconsistency

```
javascript.snippet JAVASCRIPT
Redundancy data Inconsistency data conflicting value
```

Redundancy inconsistency

Part 24 -- Atomicity

77. Atomicity ?

Atomicity ACID- property

:

Transaction ,

:

```
javascript.snippet JAVASCRIPT
All or Nothing
```

78. Bank Transfer Example

:

```
javascript.snippet JAVASCRIPT
Account A    B    500
```

A B- 200 transfer

Transaction- :

```
javascript.snippet JAVASCRIPT
A    200 +B- 200
```

operation transaction commit

failure , transaction rollback valid state-

Part 25 -- ACID Properties

79. ACID

Database transaction- property:

```
javascript.snippet JAVASCRIPT
A    AtomicityC    ConsistencyI    IsolationD    Durability
```

Atomicity

```
javascript.snippet JAVASCRIPT
All or Nothing
```

Consistency

Transaction- database valid state-

Isolation

Durability

Transaction commit database failure persistent , subject to database system guarantees

Part 26 -- Quick Revision

Django ORM

javascript.snippet

JAVASCRIPT

```
all()   objectfilter() condition   objectexclude() condition get() single objectorder_by  
(  
    sortingdistinct() duplicate result select_related() FK / OneToOne +  
    JOINprefetch_related  
(  
    ) M2M / Reverse FK + separate queriesonly() selected fields initially loaddefer()  
selected  
    fields initially deferannotate()   object-   calculated valueaggregate()   QuerySet-  
summaryvalues  
(  
    ) dictionary-like resultvalues_list() tuple/list-like resultexists() True / Falsecount  
(  
    first()   object / Nonelast()   object / Nonelatest() latest objectearliest() earliest  
objectnone  
(  
    ) empty QuerySetcreate()   objectupdate() matching rows updatedelete() matching rows  
deletebulk_create  
(  
    ) object createbulk_update()   object updateget_or_create()   get,  
createupdate_or_create  
(  
    ) update,   createselect_for_update() transaction-   row lockingraw() Raw SQL   model  
objects
```

Database Indexing

javascript.snippet

JAVASCRIPT

```
db_index=True   field indexMeta.indexes custom/composite indexesComposite Index multiple  
  
fields-   indexIndex read/query performance improve   Too many indexes write overhead +  
storage
```

DBMS

javascript.snippet

JAVASCRIPT

```
Integrity Data valid/correct rules Entity Integrity Primary key unique + non-  
nullReferential  
Integrity Foreign key valid referenceDomain Integrity Valid type/range/  
valueConsistency  
Transaction- database valid stateConstraint Data control database ruleRedundancy  
information  
multiple Inconsistency information- conflicting valuesAtomicity All or  
NothingConsistency  
Valid stateIsolation Concurrent transaction isolationDurability Committed data  
persistence
```

Final Mental Map

javascript.snippet

JAVASCRIPT

```
                                Django ORM  
  
Querying      Optimization      Aggregation  
filter  
( )      select_related()  annotate()  exclude()      prefetch_related()  aggregate()  
  
get()      only()  order_by()      defer()  distinct()  
Result  
          values()      values_list()      Checking  
  
exists()      count()      first()      last()  
  
Modification      create()      update()      delete()  
  
bulk_create()      bulk_update()      get_or_create()  
  
update_or_create()      Database  
  
Indexing      DBMS      db_index  
Integrity  
      Meta.indexes      Consistency      Composite Index      Constraint  
  
Redundancy      Inconsistency      ACID  
  
Atomicity Isolation Durability
```

```
1. ForeignKey / OneToOne    select_related()2. ManyToMany / Reverse ForeignKey
prefetch_related
()3. Per-object calculation    annotate()4. Whole QuerySet summary    aggregate()5.
Specific
fields    values() / values_list()6. Query existence    exists()7. First object safely
first()8. Multiple-field index    Meta.indexes9. Row locking    select_for_update() +
transaction
.atomic()10. Transaction "All or Nothing"    Atomicity
```